

BRAM read and write example

C 코드

```
// test data
short T[28][28] = {          //8 bit, unsigned, test data.
#include "test_data.h"
};

// node
short I[28][28];          //8 bit, unsigned

// loop index
int i, j;

// read external data
for (i=0; i<=27; i++){
    for (j=0; j<=27; j++){
        I[i][j] = T[i][j];
    }
}
```

배열 변수를 BRAM으로 구현.

Xilinx FPGA의 BRAM은 2차원 배열까지 구현가능. 2차원 주소를 사용하는 것이 복잡함.

2차원 배열을 1차원 배열로 변환함

구현용 C 코드

```
// ROM
short T[784] = {      // unsigned, width 8bit x depth 10bit, SPROM
#include "test_data.h"
};

// RAM
short I[784];      // unsigned, width 8bit x depth 10bit, SPRAM

// loop index, register
int i;      // max=783, 10bit

for (i=0; i<=783; i++){
    I[i] = T[i];
}
```

BRAM read and write

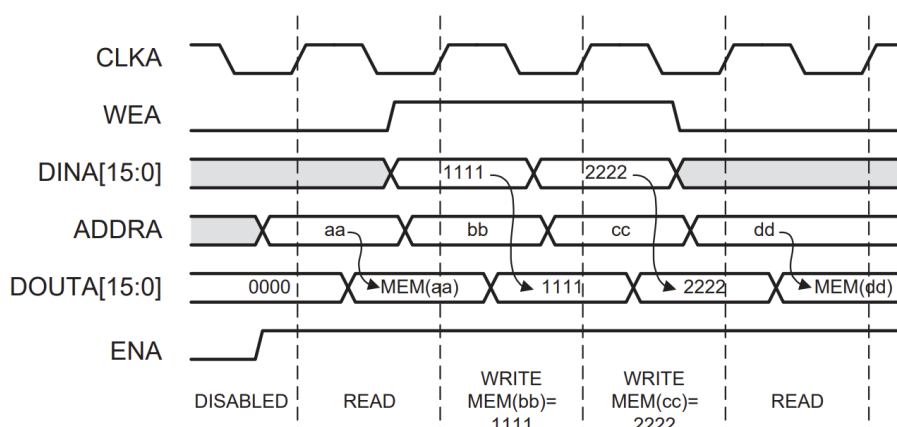


Figure 3-9: Write First Mode Example

ROM: wea, dina 포트가 없음.

Verilog code

```
module mtest(clk, rst, i_addr, i_wdata);
    input clk;
    input rst;
    output [9:0]      i_addr;
    output [7:0]      i_wdata;

    // RAM
    reg [9:0]         i_addr;    //addra
    reg [7:0]         i_wdata;    //dina
    wire [7:0]        i_rdata;    //douta
    reg               i_we;       //wea

    // ROM
    reg [9:0]         t_addr;     //addra
    wire [7:0]        t_rdata;    //douta

    // loop index register
    reg [9:0]         i;
    reg [9:0]         ni;

    // state
    localparam s0 = 0, s1 = 1, s2 = 2;

    reg [1:0]         state;
    reg [1:0]         nstate;

    // RAM instantiation
    i_ram i_test (.clka(clk), .wea(i_we), .addra(i_addr), .dina(i_wdata),
                  .douta(i_rdata));

    // ROM instantiation
    t_rom t_test (.clka(clk), .addra(t_addr), .douta(t_rdata));
```

```

always@(posedge clk)
begin
    state <= nstate;
    i    <= ni;
end

// FSM
always @ (rst, state, t_rdata, i)
begin

    // memory default
    t_addr  = 0;
    i_addr  = 0;
    i_wdata = 0;
    i_we    = 0;

    // register default
    nstate  = state;
    ni      = i;

    case (state)
    s0: begin
        nstate = s1;
        t_addr = 0;
        ni = 0;
    end // case: s0

    s1: begin
        i_wdata = t_rdata;
        i_addr = i;
        i_we = 1'b1;
        nstate = s1;
        ni = i + 1;
        if (ni == 784) begin
            nstate = s2;
        end
        t_addr = ni;
    end // case: s1

```

```

        default: begin
            nstate = s2;
        end // case: s2
    endcase

    if (rst == 1) begin
        nstate      = s0;
        ni          = 0;
    end

end

// synthesis translate_off
always@(posedge clk) begin
    if (state == s1) begin
        $display("i[%d] = %d", i_addr, i_wdata);
    end
end

// synthesis translate_on
endmodule

```

tb.v

```
//run 8000ns
```

```
module tb;
    reg        clk;
    reg        rst;
    wire [9:0]  i_adata;
    wire [7:0]  i_wdata;

    mtest uut (.clk(clk), .rst(rst), .i_addr(i_addr), .i_wdata(i_wdata));

    // 100 MHz clock
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // reset
    initial begin
        rst = 1;
        #100 rst = 0;
    end
endmodule
```

mtest.xdc

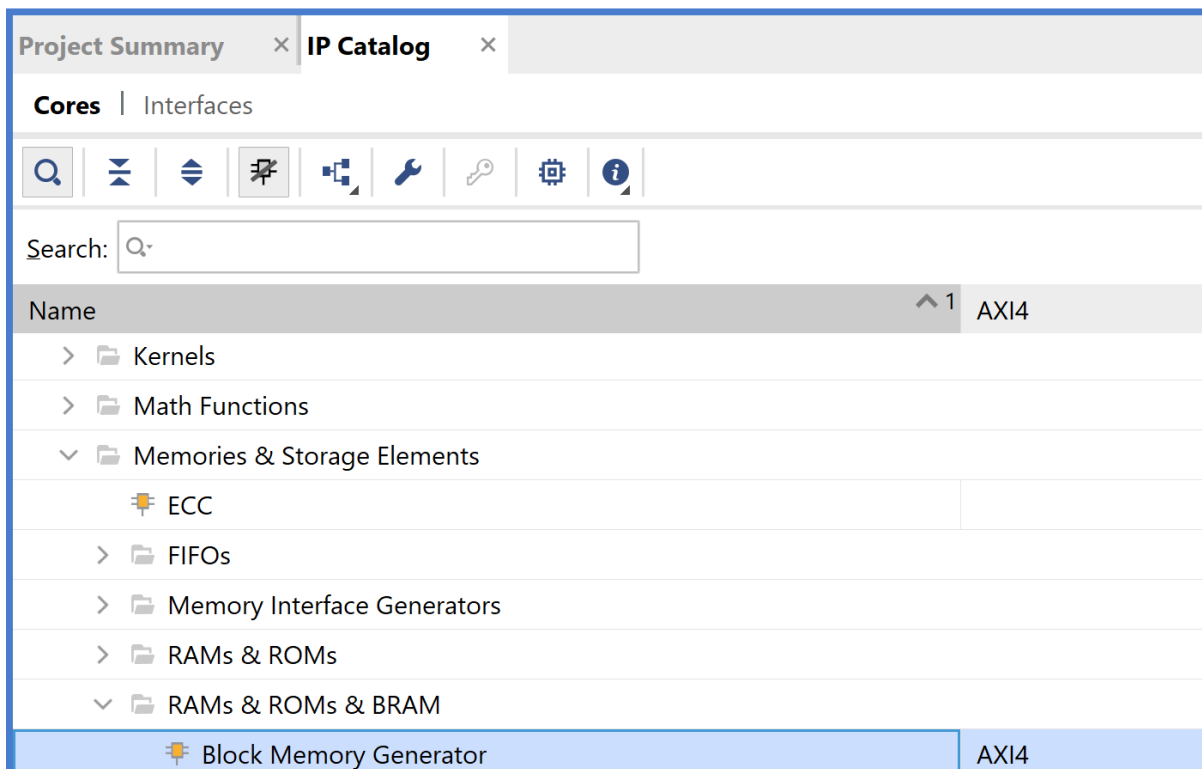
```
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]
```

RAM IP 생성

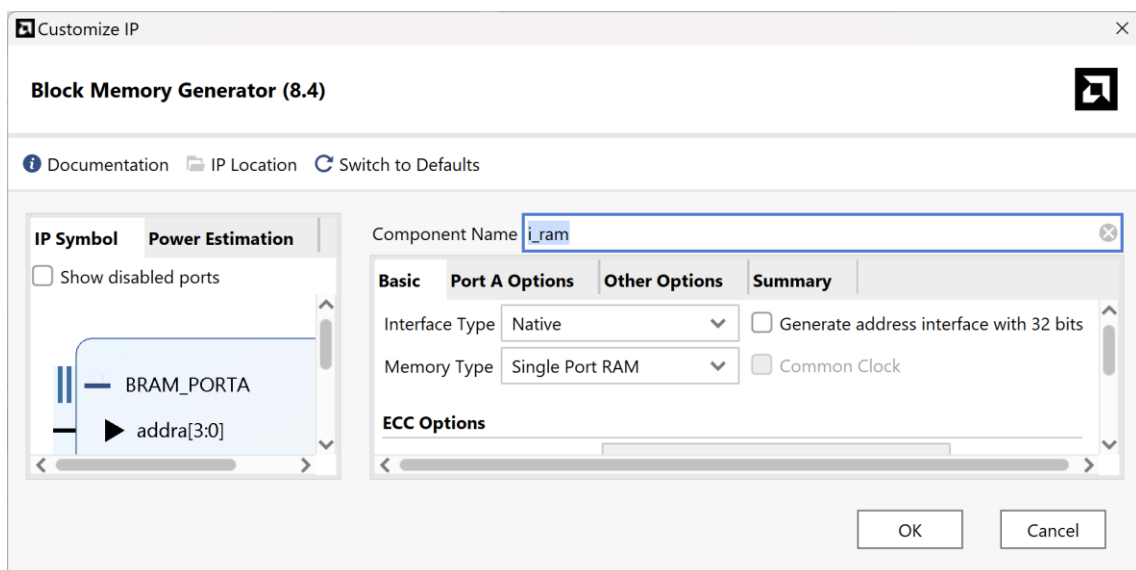
Project manager → IP catalog 클릭

IP catalog 창 → Memories & storage elements 확장 → RAMs & ROMs & BRAM 확장

Block memory generator 더블클릭



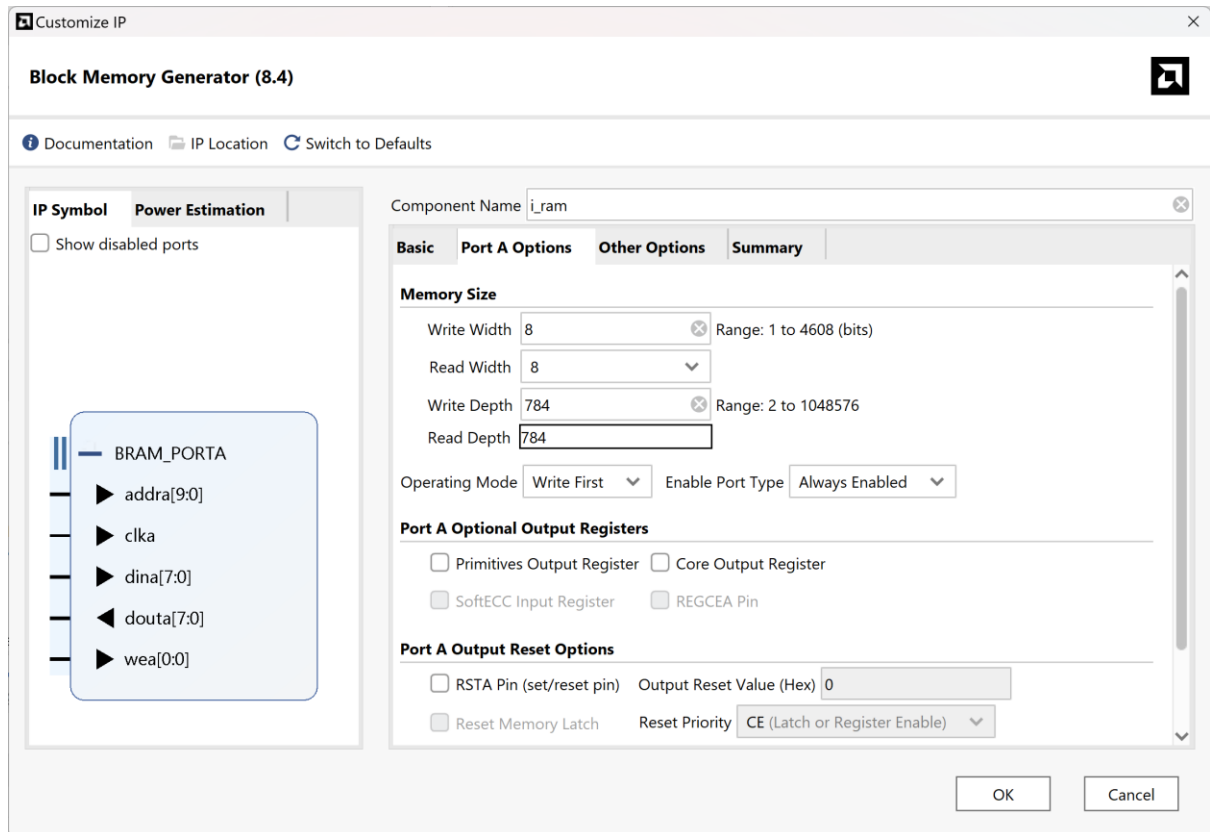
Component Name: i_ram 입력



Write width: 8

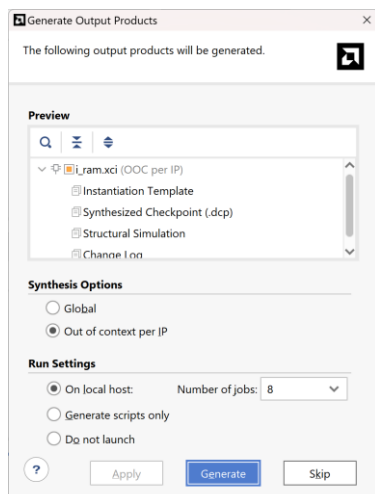
write depth: 784

그외 write first, always enabled 선택. Primitive output register 선택해제



Ok 버튼 클릭

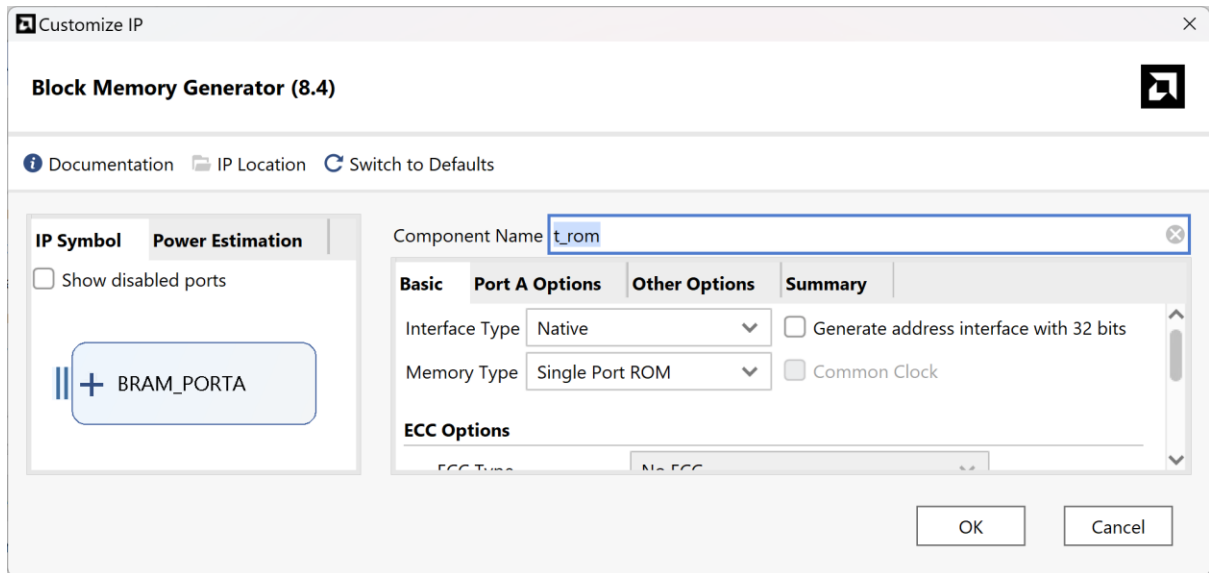
팝업창에서 generate 버튼 클릭



ROM IP 생성

Component name: t_rom 입력

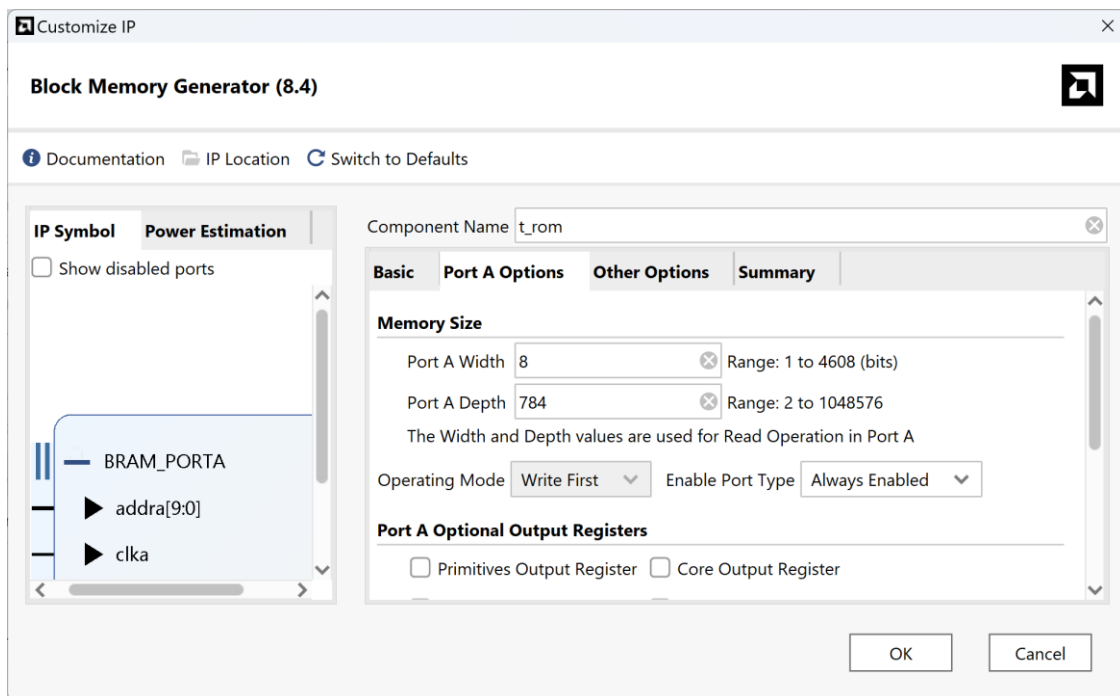
Memory type: single port ROM 선택



port A width: 8

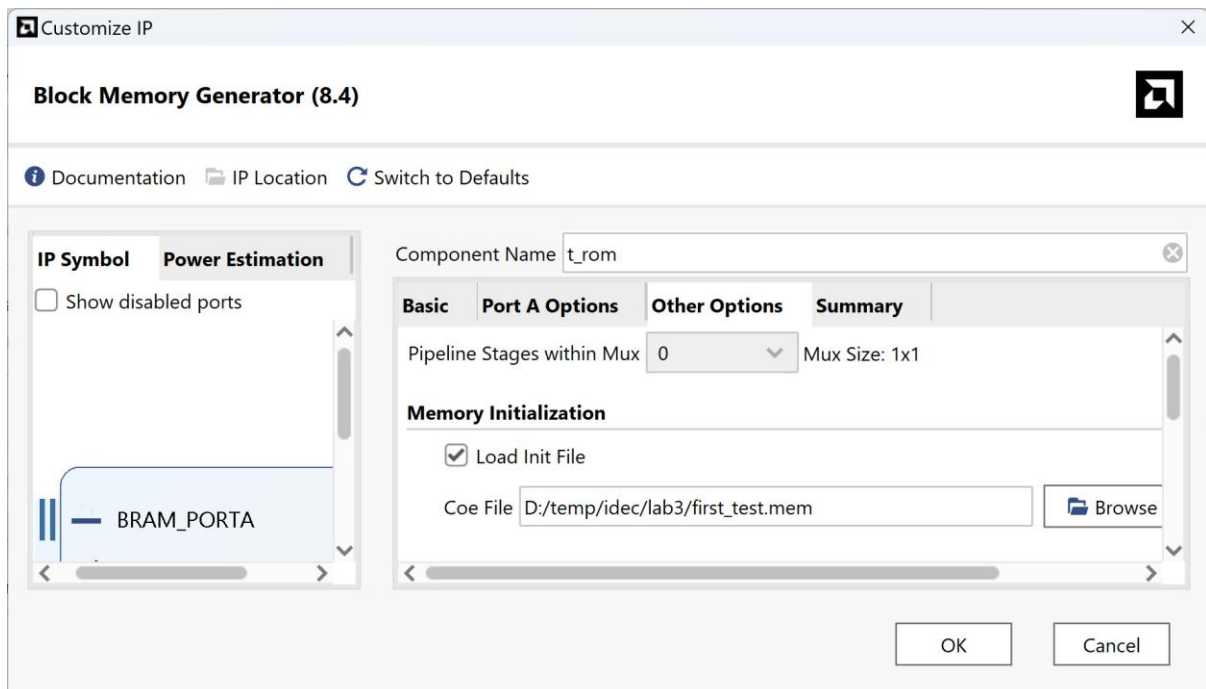
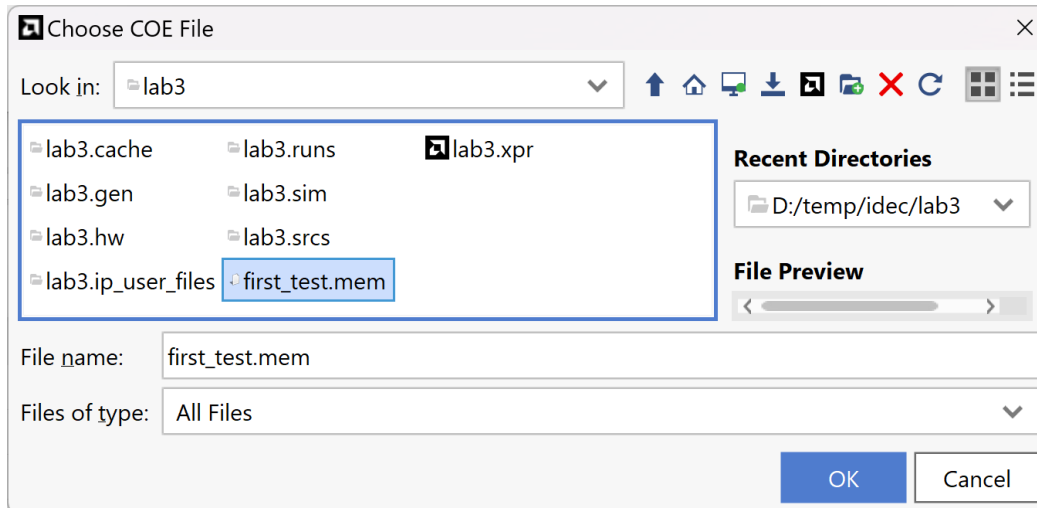
port A depth: 784

그외 always enabled 선택. Primitive output register 선택해제



Other option탭에서

Load init file 체크후 browse 버튼을 클릭해서 first_test.mem 파일선택. 이때 files of type을 all files로 할 것.



Ok 버튼 클릭후 generate 버튼 클릭